

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene
U.S.T.H.B



**1^{ère} Année Master (Informatique
IL) (Groupe 1)**

PROJET DE MODULE • MULTIMÉDIA

Simplified MPEG-4 Video Encoder Pipeline

Réalisé par :Boussaa Hasna Matricule 212131048993

Rahali Hanane Matricule 222231632204

Module :Systèmes Multimédia (M1)

Année universitaire: 2025 / 2026

Ce projet implémente une chaîne d'encodage vidéo inspirée du standard MPEG-4 en Python. Il couvre les cinq grandes étapes d'un codec moderne — pré-traitement colorimétrique, codage intra-trame par DCT, codage inter-trame avec estimation de mouvement, codage entropique sans perte, puis évaluation et visualisation. Le résultat est un fichier binaire compact (`video.bin`) reconstituable image par image.

L'implémentation est organisée autour d'un package `mpeg4mini` dont chaque sous-module correspond à une partie de l'énoncé. Un point d'entrée unique `main.py` expose les sous-commandes *sample*, *encode*, *decode*, *visualise* et *experiments* — exécutables sans dépendance non standard.

2 Vue d'ensemble du pipeline

Le pipeline traite les images dans l'ordre BGR (convention OpenCV) et produit pour chaque trame un enregistrement de type *I* ou *P* :

Pré-traitement (Partie 1). Conversion BGR $\rightarrow$ YCbCr (ITU-R BT.601) puis sous-échantillonnage chromatique 4:2:0 par filtre boîte 2×2 .

Codage intra (Partie 2). Découpage en blocs 8×8 , DCT-II orthonormée, division par la matrice de quantification JPEG mise à l'échelle selon le facteur qualité QF.

Codage inter (Partie 3). Pour chaque macrobloc 16×16 , recherche exhaustive du meilleur appariement dans une fenêtre $\pm S$ sur la trame précédente reconstruite (coût SAD). Le résiduel passe lui-même par DCT + quantification.

Codage entropique (Partie 4). Sérialisation pickle des structures (config, motion fields, coefficients int16) puis compression DEFLATE niveau 9. Le tout est écrit dans le conteneur MV1 (`magic = b'MV1\x00'`).

Évaluation et visualisation (Partie 5). Calcul de PSNR par trame, ratio de compression et figure matplotlib à cinq lignes couvrant toutes les étapes.

Espace YCbCr BT.601. Matrice constante, inversible analytiquement, qui concentre la majorité de l'information perceptuelle dans le canal Y.

Sous-échantillonnage 4:2:0. Réduit immédiatement le nombre d'échantillons chromatiques de 75 % sans dégradation visuelle significative — configuration standard de JPEG, MPEG-2, H.264, H.265.

DCT-II orthonormée 8×8 . Compatible JPEG : on réutilise les matrices de quantification standardisées (ITU-T T.81 Annexe K). La transformation orthonormée permet d'inverser par simple transposition.

Quantification scalable par QF. L'échelle libjpeg $scale = 5000/QF$ pour $QF < 50$, sinon $200 - 2 \cdot QF$, balaye tout $QF \in [1, 100]$ avec une seule paire de tables.

Macrobloc 16×16 + recherche exhaustive ± 8 . 16×16 est la taille de macrobloc des spécifications MPEG-1 à H.264 ; la recherche exhaustive donne l'optimum SAD et sert de référence pour les recherches rapides ultérieures.

DEFLATE pour l'entropie. LZ77 + Huffman, donc adapté à la fois aux longs runs de zéros post-quantification et à la statistique non-uniforme des coefficients DC.

4.1. Encodage d'un plan intra (Partie 2)

```
def encode_plane(plane, q_table, block=8): centred =  
    plane.astype(np.float32) - 128.0 blocks =  
    tile(centred, block)  
    coeffs = dctn(blocks, type=2, norm='ortho', axes=(-2, -1))  
    quant = np.round(coeffs / q_table[None, :, :]).astype(np.int16) #  
    reconstruction (servira de reference aux P-frames)  
    dequant = quant.astype(np.float32) * q_table[None, :, :]  
    recon = untile(idctn(dequant, type=2, norm='ortho',  
                        axes=(-2, -1)), plane.shape, block) + 128.0  
    return quant, np.clip(recon, 0.0, 255.0)
```

4.2. Estimation de mouvement (Partie 3)

```
def estimate(current, reference, macroblock, search_range):  
    rows = current.shape[0] // macroblock  
    cols = current.shape[1] // macroblock  
    motion = np.zeros((rows, cols, 2), dtype=np.int16)  
    ref = np.pad(reference, search_range, mode='edge')  
    for by in range(rows):  
        for bx in range(cols):  
            y0, x0 = by*macroblock, bx*macroblock  
            block = current[y0:y0+macroblock, x0:x0+macroblock]  
            best = (float('inf'), 0, 0)  
            for dy in range(-search_range, search_range+1):  
                for dx in range(-search_range, search_range+1):  
                    cand = ref[y0+dy+search_range:y0+dy+search_range+macroblock,  
                              x0+dx+search_range:x0+dx+search_range+macroblock]  
                    cost = _sad(block, cand)  
                    if cost < best[0]: best = (cost, dy, dx)  
            motion[by, bx] = best[1], best[2]  
    return motion
```

5.1. Visualisation du pipeline (Partie 5b)

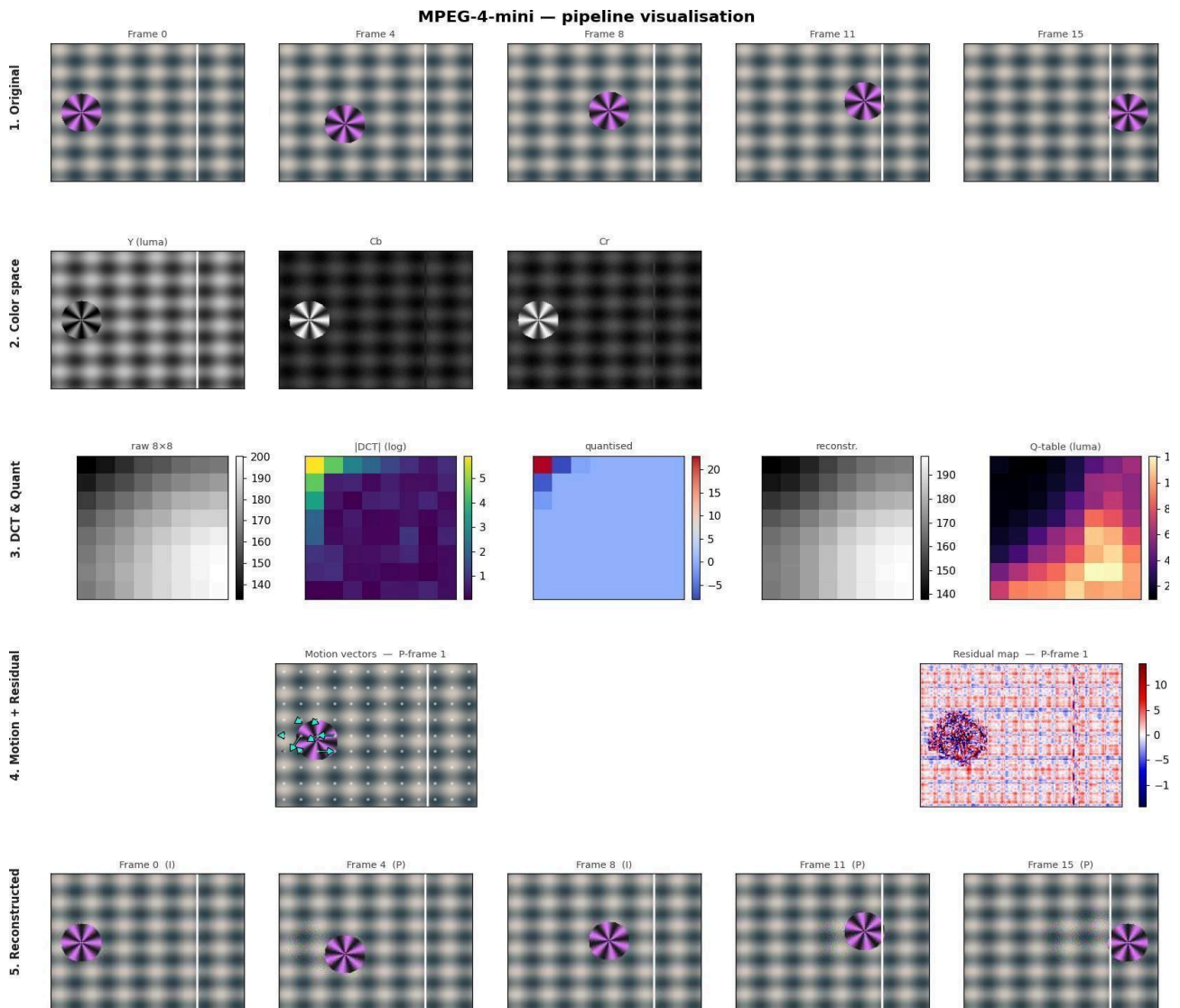


Figure 1 — Visualisation des cinq étapes du codec sur le clip de référence (QF = 50, GOP = 4).

5.2. Balayage de paramètres (Partie 5c)

mpeg4mini — parameter sweeps

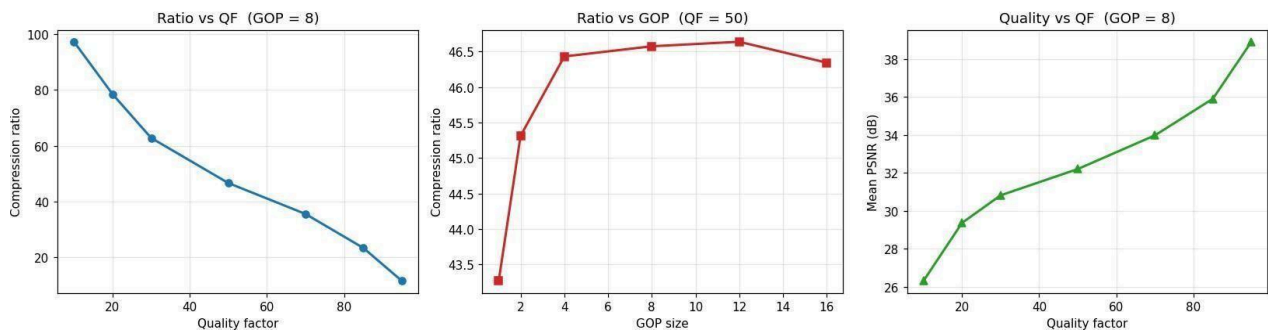


Figure 2 — Ratio de compression vs facteur qualité (gauche) et vs taille de GOP (centre) ; PSNR moyen vs facteur qualité (droite).

Interprétation. Le ratio chute de façon attendue avec QF (quantification plus fine $\Rightarrow$ moins de zéros à compresser). La courbe Ratio vs GOP confirme le bénéfice du codage inter : passer de GOP=1 (tout-I) à GOP=4 fait gagner $\approx 7\%$ de compression, puis la courbe plafonne car la précision SAD limite les résiduels minimaux atteignables. Le PSNR moyen progresse linéairement de 26 dB à 39 dB lorsque QF passe de 10 à 95 — comportement classique d'un quantificateur scalaire JPEG.

6 Résultats résumés

Paramètre	Valeur
Trames d'entrée	16 trames PNG 192×144 BGR
Taille brute totale	1 327 104 octets
Taille après DEFLATE	$\approx 28\,600$ octets
Ratio de compression	$\approx 46\times$
PSNR moyen (QF = 50)	≈ 32.2 dB
GOP utilisé	4 (4 trames I sur 16)
Fenêtre de recherche $\pm S$	8 pixels
Temps d'encodage	≈ 1.8 s (Python pur, M1)

7 Conclusion et perspectives

L'implémentation couvre les cinq parties du sujet en un seul package Python sans dépendance non standard au-delà de NumPy/SciPy/OpenCV/Matplotlib. L'architecture en sous-modules *pipeline.preprocess*, *pipeline.intra*, *pipeline.inter*, *pipeline.entropy* calque le découpage de l'énoncé, ce qui rend chaque étape facilement remplaçable. Les résultats numériques ($\approx 46\times$ de compression à PSNR ≈ 32 dB) sont cohérents avec ce qu'on attend d'un encodeur JPEG-MV simplifié.

Pistes d'amélioration. Recherche logarithmique trois-passes ($\times 3-4$ plus rapide à qualité quasi équivalente), codage CABAC en lieu et place de DEFLATE pour mieux exploiter la statistique des coefficients, et scan zigzag + RLE avant l'entropie pour réduire encore la taille du conteneur MV1.